

AI Health Advisor & Smart Hospital Recommender



SUPERVISED BY

Mr. Rasty Shaxawan Majeed
Ms. Zhwan Mohammed Khalid

PREPARED BY

Alan Aumed Najmadin

Table of Contents

Introduction 3
Project Overview 3
System Architecture 4
New Improvements 8
Future Enhancements 8
Conclusion 9
References 10









Introduction

The goal of artificial intelligence (AI) is to create machines with human-like learning, thinking, and decision-making abilities. In our last project, we developed a straightforward rule-based health advisor that provided basic health recommendations using "if-then" rules. Rule-based systems can't learn or get better on their own, even though it was useful as an example of symbolic AI.

In order to create a more intelligent and practical health support system, we advance this project by incorporating **Machine Learning (ML)**. Our new models identify patterns in data and use them to forecast risks to both **physical** and **mental** health rather than depending on predetermined rules. We also included a **Smart Hospital Recommender** that shows nearby hospitals based on the user's GPS location.

Project Overview

Our prior Health Advisor system is being continued and improved with this project. The new version adds **two** trained machine learning models, a **hospital finder**, and an **enhanced** user interface created with Streamlit, whereas the previous version solely concentrated on rule-based logic.

Currently, the system has three primary functions:

- Prediction of **Physical Health Risk**
  Based on lifestyle factors like age, sleep, water intake, and physical activity, a TensorFlow model determines whether the user's physical health risk is Low, Moderate, or High.
- Prediction of **Mental Health Risk**
  To determine the user's mental health risk, a second TensorFlow model examines social support, stress level, history of anxiety or depression, and other mental health indicators.
- **Hospital Recommendation**
  The system can determine the user's location using the Google Maps API and display hospitals that are close by within a selected radius.

All of these features are combined in the **Streamlit** interface, which allows the user to view recommendations or hospital options, enter their information, and receive predictions instantly. This turns the project into a small, working prototype of a contemporary health advisory application rather than merely a demonstration of AI concepts.

System Architecture

Our Health Advisor system is made of three main building blocks:

1. Machine Learning (ML) models
2. Hospital Finder
3. Streamlit User Interface

These parts work together to collect user data, predict health risks, and show useful results in a friendly web app.

1.Machine Learning Models:

Our system's machine learning component is in charge of evaluating user data and determining risk levels.

Two TensorFlow models are employed:

-One physical health risk model

-One mental health risk model

Both models adhere to the same concept:

- Information (age, sleep duration, stress, depression history, etc.) is entered by the user.
- These values are packed by the application into a feature vector, which is an organized format.
- The metadata saved during training is used to preprocess (scale and order) the values.
- After processing the input, the machine learning model generates probabilities for every class:

Inside the project, this logic is mainly implemented in:

- **health_model_tf.py** – physical health risk model
- **mental_model_tf.py** – mental health risk model
- Training **notebooks**:
    - 02_train_health_model_tf.ipynb
    - 03_train_mental_health_model_tf.ipynb

Source code samples:

The Physical health analyzer model definition:



```
num_classes = 3

y_train_cat = keras.utils.to_categorical(y_train, num_classes)
y_test_cat = keras.utils.to_categorical(y_test, num_classes)

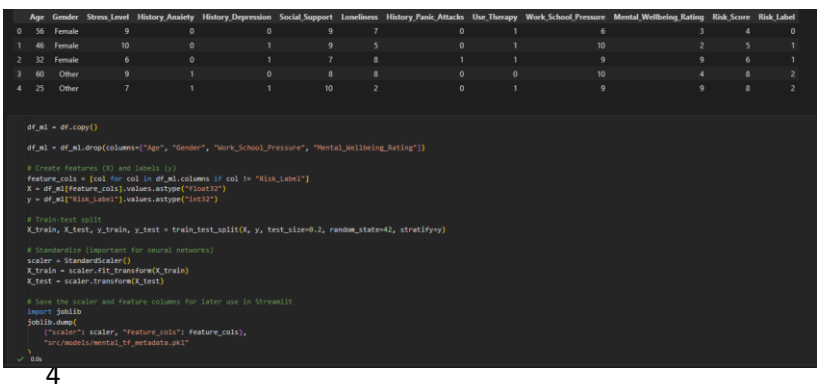
model = keras.Sequential([
    keras.layers.Input(shape=(X_train.shape[1],)),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(num_classes, activation="softmax"),
])

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)

history = model.fit(
    X_train, y_train_cat,
    validation_split=0.2,
    epochs=20,
    batch_size=32,
    verbose=1,
)

loss, acc = model.evaluate(X_test, y_test_cat, verbose=0)
print("Test accuracy:", acc)
```

The Mental Health Analyzer model:



	Age	Gender	Stress_Level	History_Anxiety	History_Depression	Social_Support	Loneliness	History_Panic_Attacks	Use_Therapy	Work_School_Pressure	Mental_Wellbeing_Rating	Risk_Score	Risk_Label
0	56	Female	9	0	0	9	7	0	1	6	3	4	0
1	46	Female	10	0	1	9	5	0	1	10	2	5	1
2	32	Female	6	0	1	7	8	1	1	9	9	6	1
3	60	Other	9	1	0	8	8	0	0	10	4	8	2
4	25	Other	7	1	1	10	2	0	1	9	9	8	2

```
df_ml = df.copy()

df_ml = df_ml.drop(columns=["Age", "Gender", "Work_School_Pressure", "Mental_Wellbeing_Rating"])

# Create features (X) and labels (y)
feature_cols = [col for col in df_ml.columns if col != "Risk_Label"]
X = df_ml[feature_cols].values.astype("float32")
y = df_ml["Risk_Label"].values.astype("int32")

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Standardize (important for neural networks)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Save the scaler and feature columns for later use in Streamlit
import joblib
joblib.dump(
    {"scaler": scaler, "feature_cols": feature_cols},
    "src/models/mental_tf_metadata.pkl"
)
```

Both of the models use datasets for their training.
Physical: **synthetic_health_lifestyle_dataset.csv** (7500 rows of data).

Mental: **mental_health_dataset.csv** (5000 Rows of data).

2. Hospital Finder

The hospital finder is the location-aware part of the system.

Its purpose is to assist the user in responding to the question, "Where can I go if my risk is high?"

This is how it operates:

- Through the browser, the user gives the app access to their current location (latitude and longitude).
- These coordinates are sent by the application to the hospital_finder.py function.
- This function uses the coordinates and a radius (e.g., 5 km) to call the Google Places API.
- In response, Google provides a list of local hospitals with the following information: Name, Address, Location (lat/lon), Rating, and additional details (if available).

This component of the system makes the project seem more realistic and helpful by tying the AI predictions to practical choices.

Hospital finder Structure:

App.py | hospital_finder.py | secrets.toml | 02_train_healt

.streamlit > secrets.toml

```
1   [google]
2   [redacted]_api_key = "[redacted]"
3
```

Here, the API key is put inside the variable, to be used inside the hospital_finder.py

```python
API_KEY = st.secrets["google"]["[redacted]pi_key"]
PLACES_NEARBY_URL = "https://maps.googleapis.com/maps/api/place/nearbysearch/json"


def find_hospitals_by_coords(lat: float, lon: float, radius: int = 5000) -> Tuple[List[Dict], Dict]:
    """Return a tuple (hospitals_list, raw_api_response).

    The raw response is returned for diagnostics so callers can show the API status
    and error messages when no results are available.
    """
    params = {
        "key": API_KEY,
        "location": f"{lat},{lon}",
        "radius": radius,
        "type": "hospital",
    }

    try:
        resp = requests.get(PLACES_NEARBY_URL, params=params, timeout=10)
        data = resp.json()
    except Exception as e:
        # Return empty list and a simple error-like dict for diagnostics
        return [], {"status": "REQUEST_EXCEPTION", "error_message": str(e)}

    status = data.get("status")
    if status not in ("OK", "ZERO_RESULTS"):
        # Return the raw data so the caller can show status / error_message
        return [], data

    hospitals: List[Dict] = []
    for place in data.get("results", []):
        hospitals.append(
            {
                "name": place.get("name"),
                "address": place.get("vicinity") or place.get("formatted_address"),
                "rating": place.get("rating"),
                "user_ratings_total": place.get("user_ratings_total"),
                "location": place["geometry"]["location"],
                "place_id": place.get("place_id"),
                "open_now": place.get("opening_hours", {}).get("open_now"),
            }
        )

    return hospitals, data
```

The API request is sent (Google's Maps Embed API and Places API).

to receive the location data and recommend the hospitals that are in that vicinity.

The name, address, rating and other information is shown.

2. Streamlit Interface

The Streamlit interface is the Main UI that the user interacts with, it's the **Face** of the system.

The **app.py** file handles that, the structure is as followed:

1) **Collect the Input**
   Display sliders, number inputs, dropdowns, and text boxes for all required fields.
   Example: age, sleep duration, stress level, history of depression, etc.

2) Call the Models

When the user clicks the **"Analyze"** button, the app:

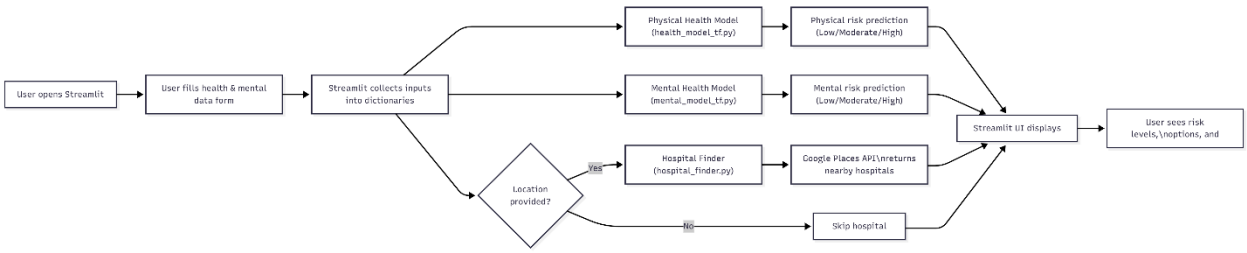
- Reads all the values from the form
- Creates Python dictionaries for physical and mental health inputs
- Sends them to the ML model classes (**`HealthRiskModelTF`, `MentalHealthRiskModelTF`**)
- Receives the risk predictions back

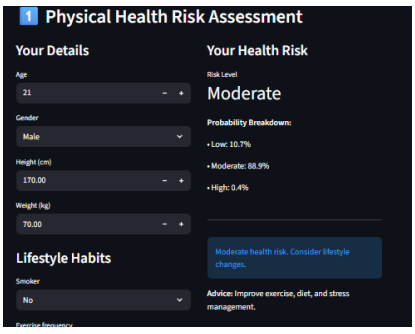
3) Call the Hospital Finder

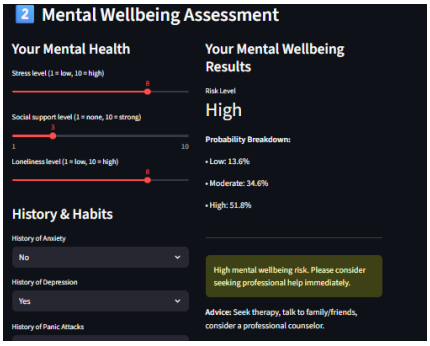
If the user shares their location, which can be done in three ways, the app calls the hospital finder to get nearby hospitals.

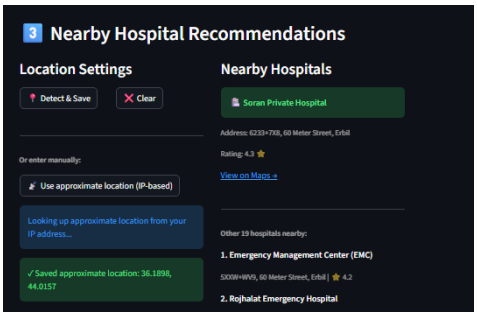
4) Display Results

- Show physical and mental health risk levels
- Show summary messages and explanations
- Show the hospital list and/or map









app.py Code Samples:

Physical Health

```python
# 1️⃣ Physical Health Risk Assessment
# ----------------------------------------

st.header("1️⃣ Physical Health Risk Assessment")

col_ph_left, col_ph_right = st.columns([1, 1], gap="medium")

with col_ph_left:
    st.subheader("Your Details")
    # Lifestyle-related features (user selects)
    age = st.number_input("Age", min_value=18, max_value=100, value=25)
    gender = st.selectbox("Gender", ["Female", "Male", "Other"])
    height_cm = st.number_input("Height (cm)", min_value=130.0, max_value=220.0, value=170.0)
    weight_kg = st.number_input("Weight (kg)", min_value=35.0, max_value=200.0, value=70.0)

    st.subheader("Lifestyle Habits")
    smoker = st.selectbox("Smoker", ["No", "Yes"])
    exercise_freq = st.selectbox(
        "Exercise frequency", ["None", "1-2 times/week", "3-5 times/week", "Daily"]
    )
    diet_quality = st.selectbox("Diet quality", ["Poor", "Good", "Excellent"])
    alcohol = st.selectbox("Alcohol consumption", ["None", "Low", "Moderate", "High"])
    chronic = st.selectbox("Chronic disease", ["No", "Yes"])

    st.subheader("Other Factors")
    stress_level = st.slider("Stress level (1-10)", 1, 10, 5)
    sleep_hours = st.slider("Sleep hours per night", 3.0, 12.0, 7.0, step=0.5)

    # BMI calculation
    bmi = weight_kg / ((height_cm / 100) ** 2)
    st.caption(f"Calculated BMI: **{bmi:.1f}**")

    # Button to analyze risk
    analyze_btn = st.button("Analyze my health risk", key="health_analyze")
```

Mental Health

```python
# ----------------------------------------
# 2️⃣ Mental Wellbeing Assessment
# ----------------------------------------

st.header("2️⃣ Mental Wellbeing Assessment")

col_mh_left, col_mh_right = st.columns([1, 1], gap="medium")

with col_mh_left:
    st.subheader("Your Mental Health")
    # Inputs for mental health (user selects)
    stress_level_m = st.slider("Stress level (1 = low, 10 = high)", 1, 10, 5, key="stress_level_
    social_support = st.slider("Social support level (1 = none, 10 = strong)", 1, 10, 5)
    loneliness = st.slider("Loneliness level (1 = low, 10 = high)", 1, 10, 5)

    st.subheader("History & Habits")
    history_anxiety = st.selectbox("History of Anxiety", ["No", "Yes"])
    history_depression = st.selectbox("History of Depression", ["No", "Yes"])
    history_panic = st.selectbox("History of Panic Attacks", ["No", "Yes"])
    use_therapy = st.selectbox("Do you currently use therapy?", ["No", "Yes"])
    work_pressure = st.slider("Work/School pressure (1 = low, 10 = high)", 1, 10, 5)

    # Button to analyze
    analyze_mental_btn = st.button("Analyze my mental wellbeing", key="mental_health_button")
```

```python
if analyze_mental_btn:
    mental_model = get_mental_model_tf()
    mental_result = mental_model.predict(mental_input)
    # Simple balanced probability adjustment
    # Start at 100% Low, then add small equal weights for each risky input factor
    # Each "risky" condition adds ~3% to the risk pool, distributed to Moderate and High

    # Count risk factors (user's "yes" answers and slider deviations from neutral)
    risk_factors = 0.0

    # History flags: each "Yes" adds 3 risk factors (increased weight for more visible impact)
    if mental_input.get("History_Anxiety", 0) == 1:
        risk_factors += 3.0
    if mental_input.get("History_Depression", 0) == 1:
        risk_factors += 3.0
    if mental_input.get("History_Panic_Attacks", 0) == 1:
        risk_factors += 3.0
```

Hospital Recommendations

```python
if lat is not None and lon is not None:
    with st.spinner("🔍 Searching..."):
        try:
            hospitals, raw = find_hospitals_by_coords(lat, lon, radius=5000)
        except Exception as e:
            st.error(f"Error searching hospitals: {e}")
            hospitals, raw = [], {"status": "EXCEPTION", "error_message": str(e)}

    if not hospitals:
        st.warning("No hospitals found nearby.")
    else:
        # Show top hospital
        h = hospitals[0]
        st.success(f"🏥 **{h['name']}**")
        st.caption(f"Address: {h['address']}")
        st.caption(f"Rating: {h.get('rating', 'N/A')} ⭐")
        st.markdown(f"[View on Maps ➔](https://www.google.com/maps/search/?api=1&query={h['location']['lat']},{h['location']['lng']})")

        # Show other hospitals in a compact way
        if len(hospitals) > 1:
            st.divider()
            st.caption(f"**Other {len(hospitals)-1} hospitals nearby:**")
            for i, hospital in enumerate(hospitals[1:], 1):
                st.markdown(f"**{i}. {hospital['name']}**")
                st.caption(f"{hospital['address']} | ⭐ {hospital.get('rating', 'N/A')}")
```

```python
# ----------------------------------------
# 3️⃣ Nearby Hospital Recommendations
# ----------------------------------------
st.header("3️⃣ Nearby Hospital Recommendations")

col_hosp_left, col_hosp_right = st.columns([1, 1], gap="medium")

with col_hosp_left:
    st.subheader("Location Settings")
    col_btn1, col_btn2 = st.columns([1, 1])
    with col_btn1:
        detect_and_save = st.button("📍 Detect & Save", key="detect_location")
    with col_btn2:
        clear_location = st.button("❌ Clear", key="clear_location")

    # Detect and save location (only when user requests)
    if detect_and_save:
        st.info("🔄 Requesting location access...")
        geo = streamlit_geolocation()
        # Save the raw geolocation result for diagnostics (even if fields are None)
        st.session_state["geolocation_raw"] = geo
        if geo and (geo.get("latitude") is not None and geo.get("longitude") is not None):
            lat = geo.get("latitude")
            lon = geo.get("longitude")
            st.session_state["geolocation"] = {"lat": lat, "lon": lon}
            st.success(f"✓ Saved: {lat:.2f}, {lon:.2f}")
        else:
            st.warning("Browser location unavailable. Use manual input below.")
```

New Improvements

Compared to the previous rule-based version, the new Health Advisor system is a significant improvement. The system is now more intelligent, useful, and comparable to real-world health applications thanks to a number of significant improvements:

1. Models for Machine Learning
Only basic **IF-THEN** rules provided suggestions in the earlier version.
Two TensorFlow models that have been trained are used by the new system:

A Model of Physical Health Risk
A Model of Mental Health Risk

2. Hospital Suggestions

The Smart Hospital Finder is a brand-new element.

The Google Maps API allows the system to:

-Find out where the user is
-Look for hospitals in the area
-Results are shown within a selected radius.
-Assist users in locating actual locations they can visit

3. Improved and Modern User Interface
The updated Streamlit user interface is now:

-Clearly arrange the sections.
-Include dynamic components
-Display forecasts, justifications, and hospital outcomes in an organized manner.
-Permit the detection of location
-Make it easier to use.

All things considered, the updated version feels more polished and comprehensive in addition to performing better.

Future Enhancements

- To increase prediction accuracy, use real medical datasets rather than fake ones.
- Develop more complex machine learning models (large architectures, ensemble techniques, or deep neural networks).
- To monitor long-term health progress, add user accounts and personal dashboards.
- Combine information from wearable technology, such as heart-rate sensors, fitness bands, and smartwatches.
- Add other health indicators like temperature, BMI, HRV, and blood oxygen.
- Include multilingual assistance in Kurdish, Arabic, English, Polish, and Turkish.

Conclusion

The improved Health Advisor system shows the shift from basic symbolic reasoning to **contemporary data-driven artificial intelligence** and is a **major** improvement over the previous rule-based method.

This new version includes **two** trained machine learning models that can recognize patterns, learn from data, and produce more accurate predictions about a user's physical and mental health, whereas the previous version could only make decisions based on set **IF-THEN rules**.

The system now operates more like an **actual** health analysis tool than a classroom demonstration, With the integration of **TensorFlow** models, preprocessing pipelines, and structured datasets. Users who receive a "moderate" or "high" risk prediction can instantly view nearby **hospitals** thanks to the addition of the **Smart Hospital Finder**, which adds a significant practical element.

By displaying results in a clear, user-friendly layout, the updated Streamlit interface further improves the user experience.

It serves as a link between the user and the AI components, integrating location-based hospital recommendations, risk assessments, and explanations into a single, coherent platform.

The system effectively demonstrates the workflow of contemporary AI-powered health applications, despite not being designed to offer actual medical diagnoses. It demonstrates to users how data is gathered, processed, analyzed, and visualized. It also shows how various technologies, **such as web interfaces, APIs, and machine learning models**, can collaborate to create a comprehensive and interactive system.

More importantly, this project demonstrates how AI systems adapt to their objectives.

What started out as a straightforward rule-based expert system has developed into a mini-ecosystem with geolocation services, automated predictions, trained models, and an intuitive user interface.

The real-world development of AI applications in healthcare is reflected in this evolution, which begins with simple decision rules and progressively incorporates more sophisticated and intelligent algorithms.

The system offers a solid basis for **future development** with numerous opportunities for enhancements, including the use of real datasets, multilingual support, IoT sensor connections, and long-term user tracking.

In its current state, the new Health Advisor is a comprehensive, useful, and instructive illustration of how machine learning and user-centered design can combine to produce a useful AI application.





References

1.
Esteva, A., Robicquet, A., Ramsundar, B., Kuleshov, V., DePristo, M., Chou, K., Cui, C., Corrado, G., Thrun, S. and Dean, J., 2019. *A guide to deep learning in healthcare*. Nature Medicine, 25(1), pp.24–29.

2.
Shickel, B., Tighe, P.J., Bihorac, A. and Rashidi, P., 2018. *Deep EHR: A survey of recent advances in deep learning techniques for electronic health record (EHR) analysis*. IEEE Journal of Biomedical and Health Informatics, 22(5), pp.1589–1604.

3.
Razzak, M.I., Imran, M. and Xu, G., 2019. *Big data analytics for preventive medicine*. Neural Computing and Applications, 32, pp.16055–16073.

4.
Kessler, R.C., Amminger, G.P., Aguilar-Gaxiola, S., Alonso, J., Lee, S. and Ustün, T.B., 2007. *Age of onset of mental disorders: A review of recent literature*. Current Opinion in Psychiatry, 20(4), pp.359–364.

5.
Chen, Z., Fang, H., Li, H. and Hu, B., 2020. *Wearable sensors for mental health monitoring*. Frontiers in Neuroscience, 14, p.14.